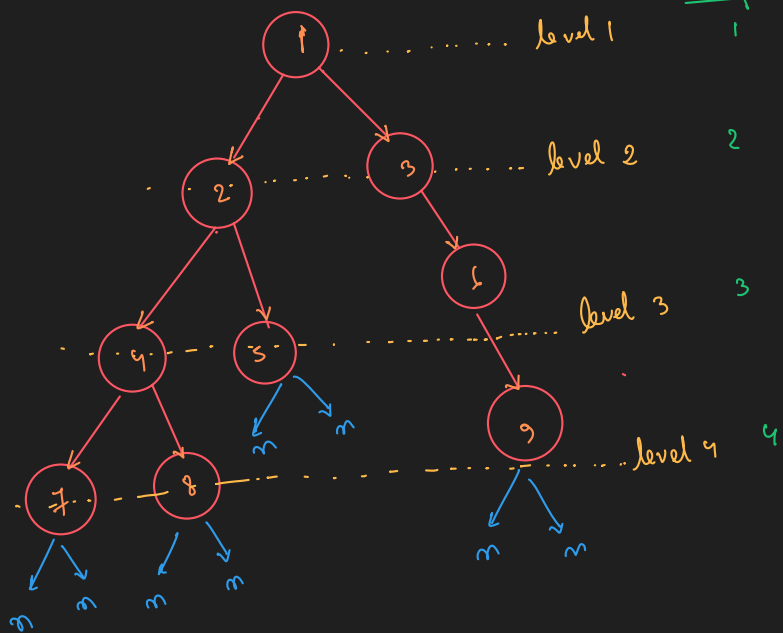
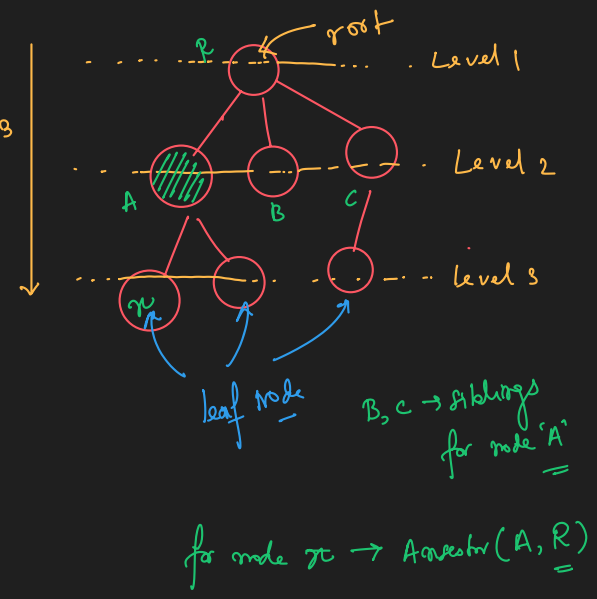


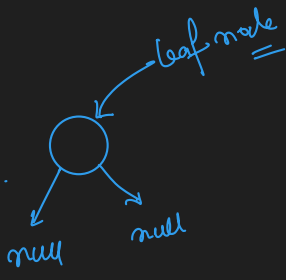
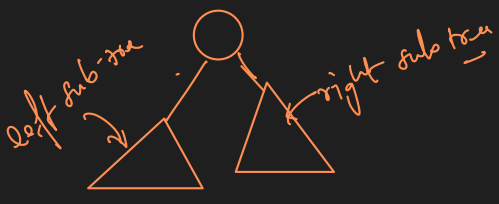
Binary tree



depth 1
depth 2
height = 3



height = 4



class Node {

int data;

Node left, right;

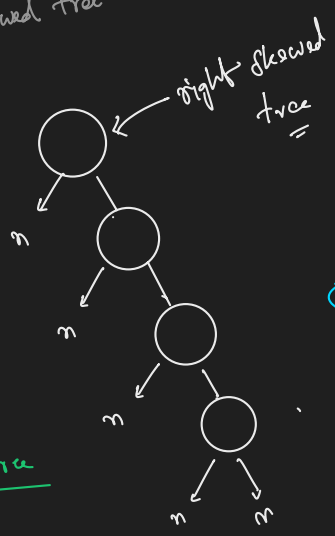
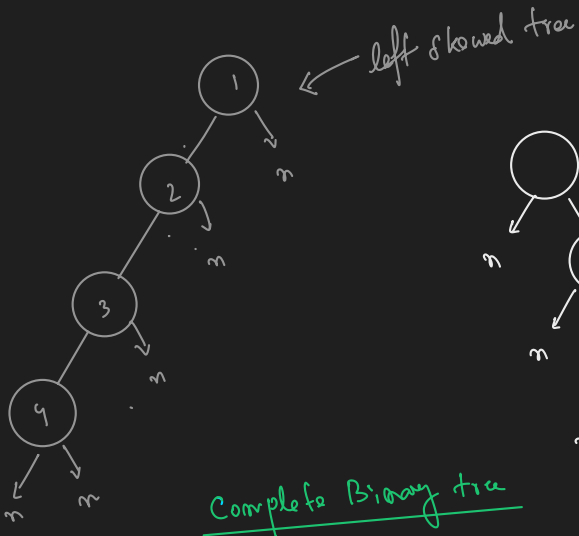
Node (int data) {

this.data = data;

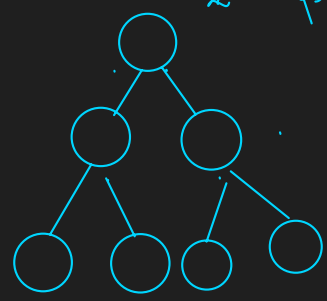
this.left = this.right = null;

}

degree of each node = 2 except the leaf node
perfect Binary tree



l = 3
d = 3



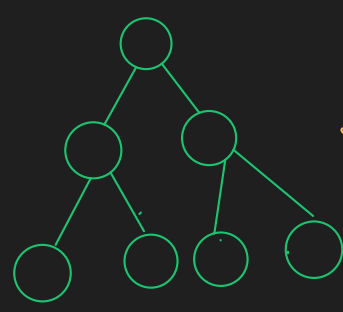
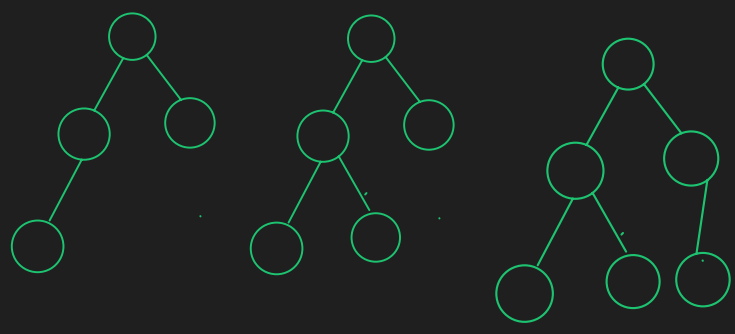
total no of nodes

$$= 2^3 - 1$$

$$= 2^l - 1$$

degree = no of sub nodes

Complete Binary tree

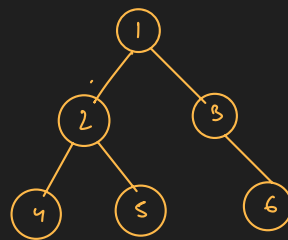
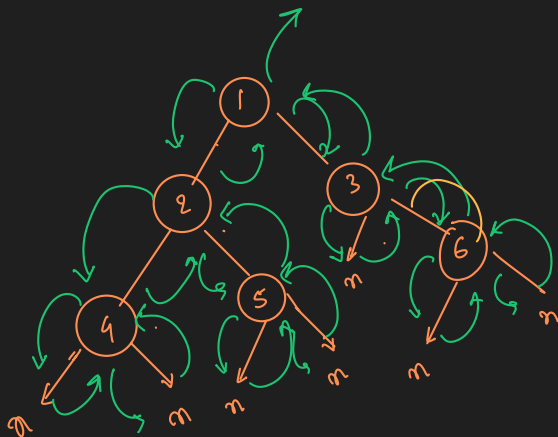


[right subtree will not be there before left subtree/node and fills the node left to right] ← for a complete level

Q Build tree Preorder

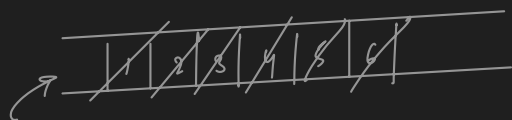
if nodes[] = { 1, 2, 4, -1, -1, 5, -1, -1, 3, -1, 6, -1, -1 }

Pre-order Sequence
[root, left, right]



1
2 3
4 5 6

level order



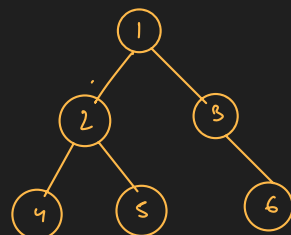
o/p: 1 2 3 4 5 6

```

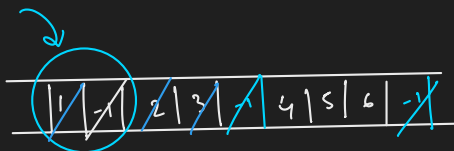
levelOrder (root) {
    if (root == null) {
        q.add (root);
    }
    while (!q.isEmpty()) {
        Node curr = q.remove();
        cout << curr.data;
        if (curr.left != null) {
            q.add (curr.left);
        }
        if (curr.right != null) {
            q.add (curr.right);
        }
    }
}
    
```

```

public static void levelOrder (Node root) {
    Queue<Node> q = new LinkedList<>();
    if (root == null) {
        return;
    }
    q.add (root);
    while (!q.isEmpty()) {
        Node curr = q.remove();
        System.out.print (curr.data + " ");
        if (curr.left != null) {
            q.add (curr.left);
        }
        if (curr.right != null) {
            q.add (curr.right);
        }
    }
}
    
```



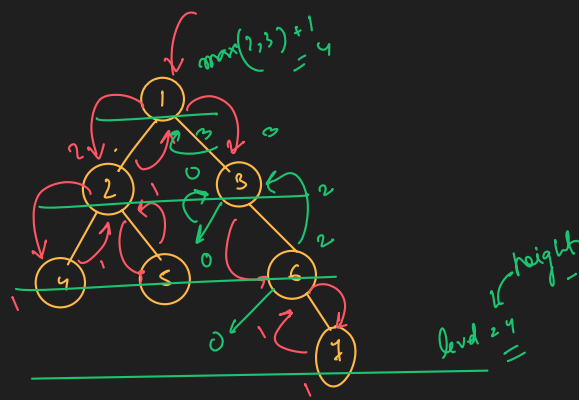
initial standing



o/p: 1
2 3
4 5 6

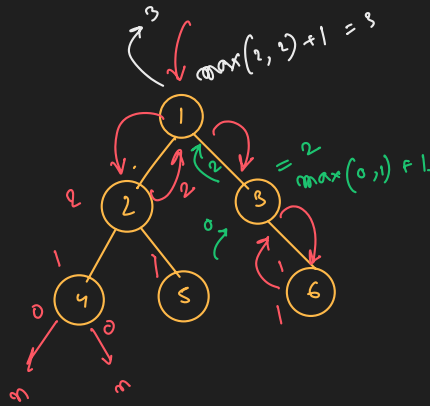
height of a tree

Time Complexity?



$$\text{height} = \max(\text{lh}, \text{rh}) + 1$$

null(node)
↳ height = 0

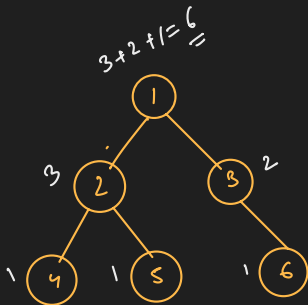


```
public static int height(Node root){
    if(root == null){
        return 0;
    }
    return Math.max(height(root.left), height(root.right)) + 1;
}
```

left height

right height

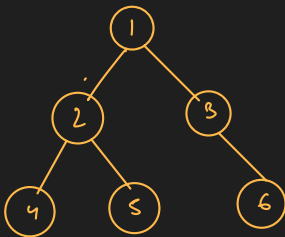
Count of the nodes



$$\text{count} = \text{left count} + \text{right count} + 1 \quad \leftarrow \text{itself}$$

```
public static int count(Node root){
    if(root == null){
        return 0;
    }
    return count(root.left) + count(root.right) + 1;
}
```

Sum of the nodes

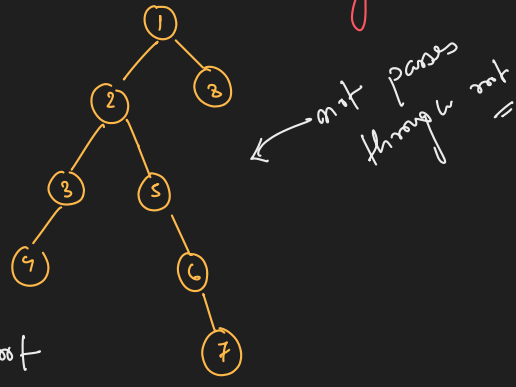
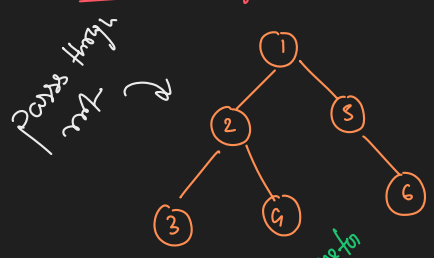


$$\text{Sum} = \text{left sum} + \text{right sum} + \text{root.data}$$

```
public static int sum(Node root){
    if(root == null){
        return 0;
    }
    return sum(root.left) + sum(root.right) + root.data;
}
```

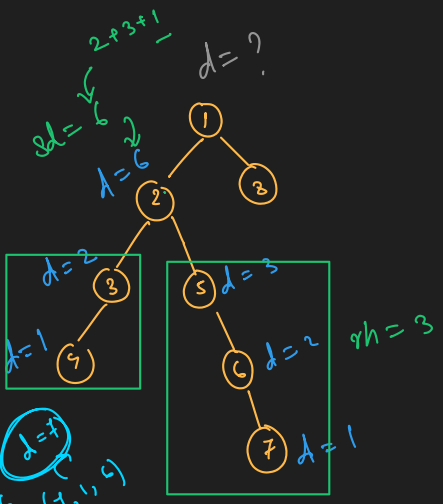
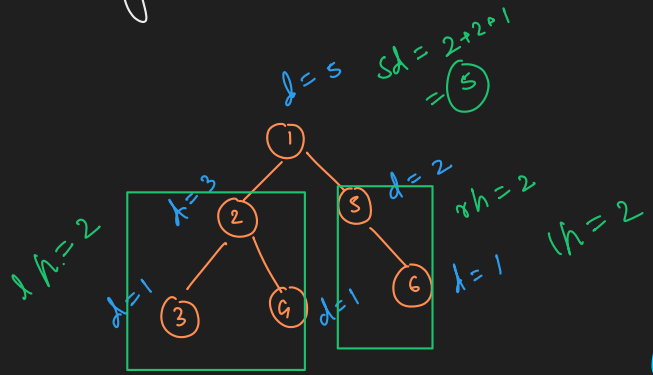
day-35

Diameter of a tree : no of nodes in the longest path b/w 2 leaves =



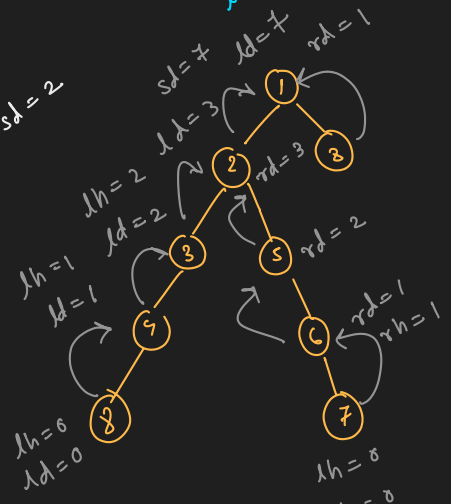
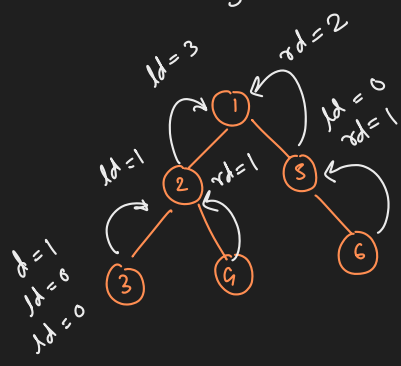
path of diameter passes through root (self) =
Self diameter = $lh + rh + 1$

diameter may pass through root
or diameter may not pass through root.



$\max(2, 2, 5) = 5$
 $sd = 2 + 2 + 1 = 5$

Assumption ①
diameter pass through self(root)



Assumption ②
diameter passes through a left or right sub tree.
→ left diam. (ld)
→ right diam. (rd)

- any node
- ① $lh = 0 = 1$
 - ② $rh = 0 = 1$
 - ③ $sd = lh + rh + 1 = 1 + 1 = 2$
 - ④ $ld = 0 = 1$
 - ⑤ $rd = 0 = 1$
 - ⑥ compute (ld, rd, sd) → return the max.

∴ diam. of tree = $\max(ld, rd, sd)$

```
public static int height(Node root){
    if(root == null){
        return 0;
    }
    return Math.max(height(root.left), height(root.right)) + 1;
}
```

$O(n)$

```
public static int diameter(Node root){
    if(root == null){
        return 0;
    }

    int lh = height(root.left);
    int rh = height(root.right);

    int selfDiam = lh+rh+1;

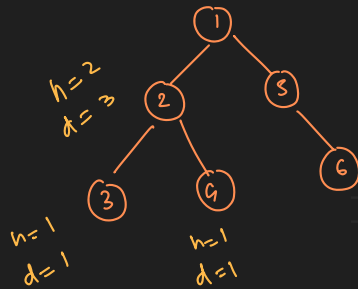
    return Math.max(selfDiam, Math.max(diameter(root.left), diameter(root.right)));
}
```

f.c = $O(n^2)$

no of nodes = n
 $\Rightarrow$ height = $O(n)$

```
class Info {
    int height;
    int diameter;
}
```

for each $\rightarrow$ return <Info>



```
static class Info {
    int height;
    int diameter;

    Info(int height, int diameter){
        this.height = height;
        this.diameter = diameter;
    }
}
```

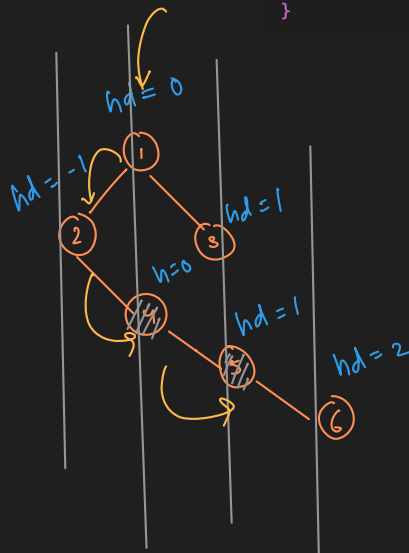
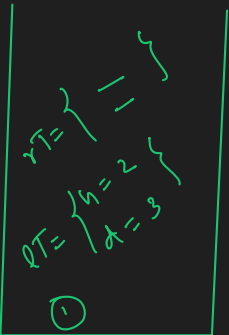
$\rightarrow$ f.c = $O(n)$

```
public static Info diameter2(Node root){
    if(root == null){
        return new Info(height: 0, diameter: 0);
    }

    Info leftTree = diameter2(root.left);
    Info rightTree = diameter2(root.right);

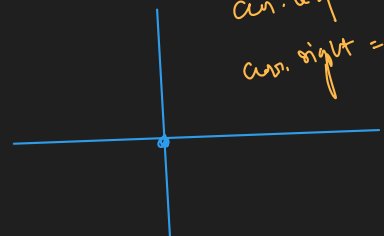
    int h = Math.max(leftTree.height, rightTree.height)+1;
    int sd = leftTree.height + rightTree.height + 1;
    int d = Math.max(Math.max(leftTree.diameter, rightTree.diameter), sd);

    return new Info(h, d);
}
```



Print : [2, 1, 3, 6]

curr node $\rightarrow$ hd
 curr. left $\rightarrow$ hd = hd - 1
 curr. right = hd + 1



Top view of a Tree
 ① calculate horizontal distance
 $\rightarrow$ key
 map (hd, node)

Hashmap (key, value) map
unique

- ① map.put (key, value) ;
- ② map.get (key) → return value (if exist)
 → return null (if doesn't exist)

Integer (hd) →

Node →

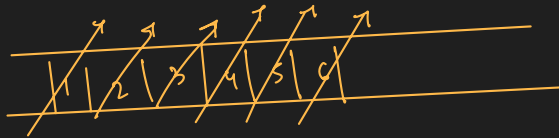
key	value
0	①
-1	②
1	③
2	④

X

if (map.get(hd) != null)
 map.put (hd, node)

if value exist don't do any update.

- ① BFS + ② hd (update) + ③ map (update)



(left tree) min = 0 - 1
 (right tree) max = 0 + 2

for (i = min to max) {
 print (map.get(i)) ;
 }

map →

hd	Node
0	①
-1	②
1	③
2	④

